Faculdade de Engenharia da Universidade do Porto



Weather App: Forecast & Analysis for Cities

'MEIC - Computação Móvel'

Group 04

João Pedro Antão Sobral (up202108736) Participation: 33.3%

João Pedro Moreira Costa (up202108714) Participation: 33.3%

Rafael Neves Teixeira (up202108831) Participation: 33.3%

Index

1. Summary **3**
2. Introduction **3**
3. System Overview **3**
4. City Management **5**
5. City Weather View **5**
6. Week Stats **5**
7. Hourly Forecast **6**
8. Current Location Weather **6**
9. Autocomplete and Suggestion mechanism **6**
10. Settings Page **7**
11. External Service Integration **7**
12. UI & User Flow **8**
13. Architecture **9**
14. Testing **9**
15. Local Database Tables **10**
Table: cities 10
Table: settings 10
16. External Libraries Used **11**
17. How to Run the Application **11**
18. Conclusion **11**
19. References **12**
20. Appendix **14**
UI App Flows 14

1. Summary

This project presents a Flutter-based weather application designed to help users monitor real-time and forecasted weather conditions across multiple cities. The application offers comprehensive weather insights, including current conditions and next-day forecasts, as well as detailed hour-by-hour breakdowns. Users can explore weekly weather trends through intuitive graphical representations, enabling easy tracking of temperature, humidity, and other key metrics over time.
The app allows users to manage a personalized list of cities, which can be populated via a convenient search bar equipped with autocomplete and suggestion features. Each city displays weather icons and textual descriptions to enhance clarity. Furthermore, the app supports GPS-based location tracking, enabling users to view weather data for their current position. To ensure a richer experience, the application also includes weather alerts and a visual depiction of sunrise and sunset times.

2. Introduction

This application was developed to provide users with a personalized and intuitive weather tracking experience. By integrating the Visual Crossing Weather API, it offers real-time data, daily forecasts, and rich graphical visualizations of weather trends. The app supports tracking conditions for multiple cities, allows customization based on user preferences, and utilizes the device's geolocation capabilities to deliver location-specific weather updates seamlessly.

3. System Overview

Main Components:

1. **Flutter Mobile App:**
    - UI for city weather consultation
    - List management (add/remove cities)
    - Live location support
2. **Weather API (Visual Crossing):**

- Provides real-time & historical weather data
- JSON responses with icons, forecasts, and hourly breakdowns

3. **Graphing Library:**
   - Displays weekly weather trends (temperature, humidity, etc.)

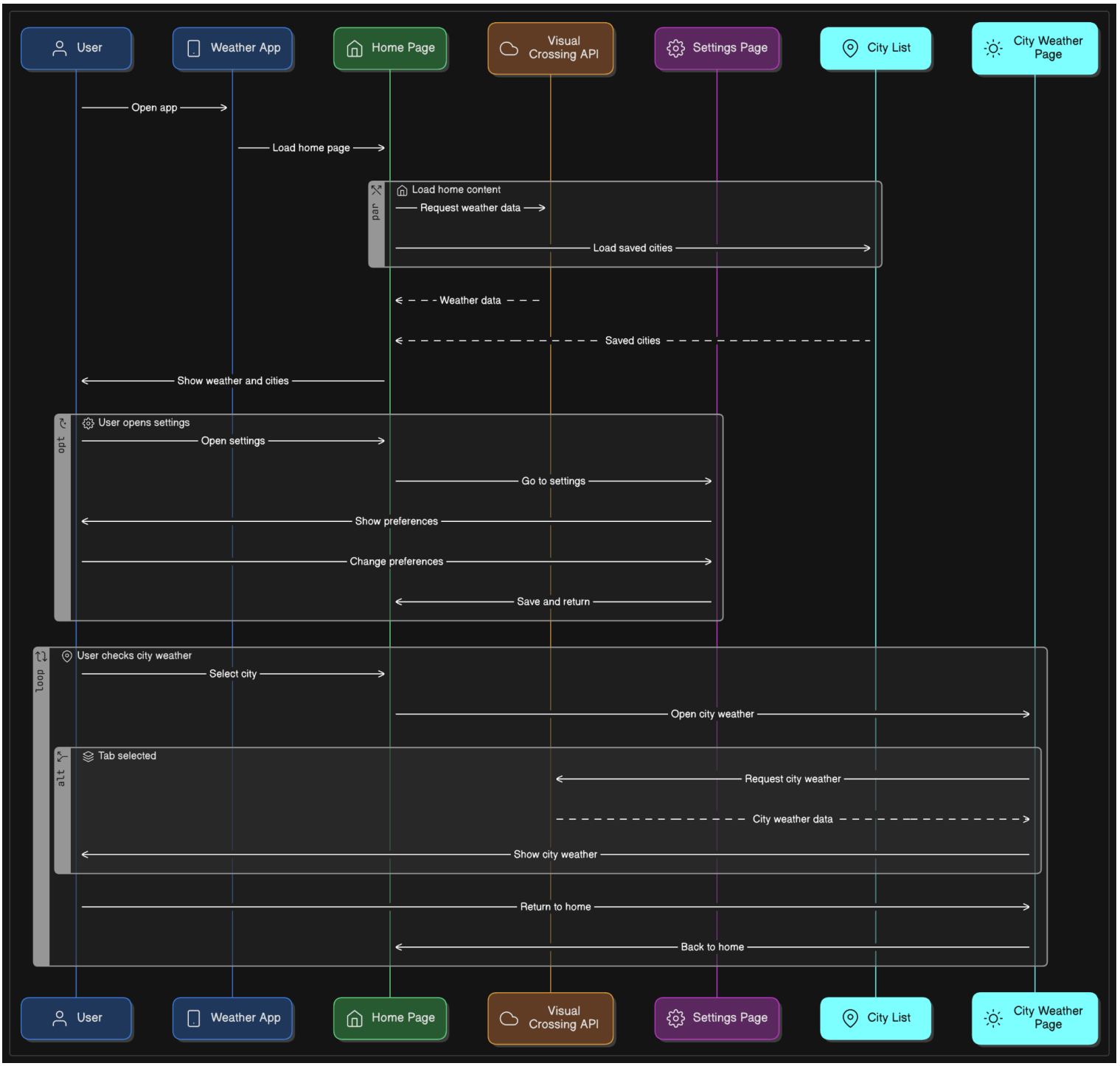


Fig. 1 - Diagram displaying the overall control and management of our main app components

4. City Management

Users can:

- Add cities with a system of auto completion and results suggestions (Fig. 7).
- Remove cities at any time (Fig. 9).
- Access weather data for saved cities (Figs. 10, 12, 14).

5. City Weather View

Each selected city has three main views:

- Current Weather (Fig. 10): Temp, humidity, wind, pressure, etc. with weather icon, weather alerts for that day and a visual demonstration of when the sun rises and sets.
- Next Days Forecast (Fig. 12): By default, the next day is selected, however, the user can select another of the 7 following days. This section includes a summary with hourly breakdown and corresponding icons, weather alerts for that day and a visual demonstration of when the sun rises and sets.
- Week Stats (Fig. 14): A graphical representation of the weather evolution throughout the week. To better understand how this was implemented, we speak about this in more detail in chapter 6.

6. Week Stats

The Week Stats feature is a key component of our weather application, offering users a graphical overview of weather trends across several days. This visualization enables the user to analyze weather metrics such as **temperature, humidity, wind speed, pressure, and precipitation** over time.

We used the **fl_chart** package to render interactive line charts. The implementation includes dynamic data handling and user input through two dropdowns:

- **Day Range Selector** – lets users choose between 1 to 7 days to be displayed on the chart
- **Metric Selector** – allows the selection of a specific weather characteristic

The line chart responds in real-time to these user inputs, redrawing the graph to reflect updated data. Each data point on the chart is plotted using the **FlSpot** structure, with customized SideTitles to render:

- The corresponding date on the X-axis
- The formatted metric value on the Y-axis (with appropriate unit conversion based on user settings)

7. Hourly Forecast

This additional feature, depicted in Fig. 4, allows the user to:

- View hourly weather for the selected day.
- Clicking a tile updates the weather card to display detailed information for the selected timestamp.

8. Current Location Weather

The application uses the device's GPS to automatically detect the user's current location. This geolocation data is then used to request weather information from the API, allowing the app to fetch and display accurate, real-time weather conditions for the user's precise location. This feature can be seen in Fig. 2.

9. Autocomplete and Suggestion mechanism

When users begin typing to search for and save a city, a dropdown menu appears with dynamic suggestions based on their input. To implement this autocomplete functionality, we initially explored using external services such as the GeoDB Cities API and Google Places API, both of which

support autocomplete features. However, acquiring an API key for these services required creating an account and providing credit card details, something we preferred to avoid.

As an alternative, we opted to use a publicly available dataset from Kaggle (https://www.kaggle.com/datasets/okfn/world-cities), which permits usage under the terms of [Creative Common Attribution License]. After downloading the dataset, we leveraged the **csv** package to read and process the data locally. The frontend implementation was completed using Flutter's built-in **Autocomplete** widget, allowing us to provide a smooth and responsive search experience without relying on third-party APIs. This feature can be seen in Fig.7.

10. Settings Page

In this page, depicted in Fig. 17, the user can pick his preferred temperature, distance and speed units. Also, he can choose if he wants the switch button (Fig. 3), used to display additional information on the weather card, to already be enabled. Whenever the user clicks on "Save Settings", the changes are applied instantly across the app in real-time and the preferences are stored in the local database.

11. External Service Integration

Visual Crossing API:

- URL structure for data request: *https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline/{city}/{date}?include=days,hours,current&key=API_KEY&iconSet=icons2;*
    - **city:** used for sending the location information
        - Example: "Porto,PT" would request weather for Porto, Portugal
    - **date:** used for retrieving weather information based on the date given
        - Example: 2024-05-21 would request information for that day
    - **?include=days,hours,current:** for receiving daily summaries (e.g., highs/lows, sunrise/sunset), Hour-by-hour forecasts and Real-time weather data, respectively
    - **&key=$_apiKey:** our API key, which authenticates the request.

- **&iconSet=icons2:** Specifies the icon set used in the response. We opted for the second icon set as we found it prettier.

Weather Icons:

- Pulled from GitHub repository.
- These icons are matched with the icon field in the API's JSON response, which corresponds to the icon set specified in the request using the **&iconSet=icons2** parameter.

12. UI & User Flow

Home Page:

- Current location weather
- Hour-by-hour view
- Weather Alerts
- Saved city list

City Detail: Tabs for:

- Current
- Next days
- Weekly graph
- The “Current” and “Next days” tabs include:
  - Hour-by-hour view
  - Weather Alerts
  - Visual illustration of when the sun rises and sets

Settings Page:

- Pick preferred metric units
- Select to start with the Switch button enabled/disabled in the weather card

Add City: Autocomplete dropdown with validation

Location Access: Permission request on app start

13. Architecture

Client-Side Only (No Backend):

- All requests made directly to Visual Crossing API
- Lightweight, responsive design using Flutter

State Management:

- Implemented with Provider package for efficiency
- Separation between UI and logic layers

14. Testing

Functional Tests:

- Add/remove cities
- Fetch current & next day forecasts
- Parse hourly and weekly data
- Display graphs accurately
- Permission handling for geolocation
- Trying to add a city with empty values
- Testing if app state breaks when accessing weather data from different hours in both sections “Today” and “Next Days”
- Checking when user picks another metric unit in the settings, if the weather metrics displayed are updated
- Tried gaining access to the database through the search bar, however, since we use parameterized queries, safety is ensured

Edge Cases:

- Invalid API key: API doesn’t send the necessary information
- No internet connection: No weather data is retrieved from the API
- Location permission denied: A message warns the user to allow permissions to display the current location weather.

Since the only potential entry point for accessing private data in our app is the search bar, and it operates entirely on client-side filtering without direct interaction with the database using raw queries, we can confidently conclude that the app is secure against SQL injection and other attacks.

15. Local Database Tables

Table: cities

- **Purpose:** Stores the list of user-selected cities.
- **Schema:**

Column	Type	Description
id	INTEGER	Primary key, auto-incremented
name	TEXT	City name (unique)

- Functions:
    - **getCities():** Returns all saved cities
    - **addCity(city):** Adds a city (ignores duplicates)
    - **removeCity(city):** Removes a city from the list

Table: settings

- **Purpose:** Stores key-value configuration data (e.g., whether the user prefers celsius or fahrenheit units).
- **Schema:**

Column	Type	Description
key	TEXT	Setting name (PK)
value	TEXT	Value of the setting

- **Functions:**
    - **setSetting(key, value):** Stores or updates a setting
    - **getSetting(key):** Retrieves the setting's value

16. External Libraries Used

- **http:** API requests
- **provider:** State management across widgets
- **geolocator:** Get user's current location (GPS access)
- **geocoding:** Converts latitude/longitude into a city name, used with geolocator (Reverse geocoding)
- **fl_chart:** Weekly graph rendering
- **sqflite:** Storing user preferences and saved cities locally in a lightweight SQL database.
- **csv:** Allows to load the csv file that contains a register of twenty-thousand entries of different countries necessary for the autocomplete search feature.

17. How to Run the Application

1. Run the application via Android Studio or command line:
   ```
   flutter run
   ```
2. Allow location permission

18. Conclusion

This weather application showcases the integration of real-time data APIs, Flutter's UI capabilities, and state management to deliver a complete and interactive weather-tracking solution. The addition of user-selected cities, hourly forecasts, and weekly trends makes it versatile and user-friendly. During development, one major challenge was ensuring UI responsiveness during asynchronous API calls. We addressed this by integrating loading indicators and optimizing state updates.

19. References

1. Classes Slides

2. Flutter Documentation.

Android Developers. https://docs.flutter.dev/get-started/flutter-for/android-devs

Official reference for our application development.

3. Dart Documentation.

Dart Documentation. https://dart.dev/docs

Official reference for our application development.

4. http Documentation.

http Documentation. https://pub.dev/packages/http

Documentation for implementing http requests in our project.

5. provider Documentation.

provider Documentation. https://pub.dev/packages/provider

Documentation that we used to manage states across our app.

6. geolocator & geocoding Documentation.

geolocator Documentation. https://pub.dev/packages/geolocator

geocoding Documentation. https://pub.dev/packages/geocoding

Documentation that we used to retrieve the users current location.

7. fl_chart Documentation.

fl_chart Documentation. https://pub.dev/packages/fl_chart

Documentation that we used to use charts in our app.

8. sqflite Documentation.

sqflite Documentation. https://pub.dev/packages/sqflite

Documentation that we used to store user saved cities and preferences locally.

9. csv Documentation.

csv Documentation. https://pub.dev/packages/csv

Documentation that we used to read and process the csv file that contains 20000 entries of world-wide cities in order to implement the autocomplete and suggestion feature.

10. GeoDB Cities & Places API

GeoDB Cities Autocomplete Mention.

http://geodb-cities-api.wirefreethought.com/docs/guides/getting-started/use-cases

Places API Autocomplete Mention.

https://developers.google.com/maps/documentation/places/web-service/legacy/autocomplete

Guides for API's that we didn't use but initially considered for implementing the autocomplete search mechanism.

20. Appendix

UI App Flows

When a user enters the app, this is the first page he sees, the Home Page:



Fig. 2 - Home Page

On this page, the user can see the weather information for his current location. By clicking on the Switch Button “Show more info”, a list with more detailed information appears:



Fig. 3 - Switch Button to display more detailed weather information based on the selected date

Still on the Home Page, below the weather card displaying the weather information, the user has a draggable row with hourly weather information for the rest of the day:

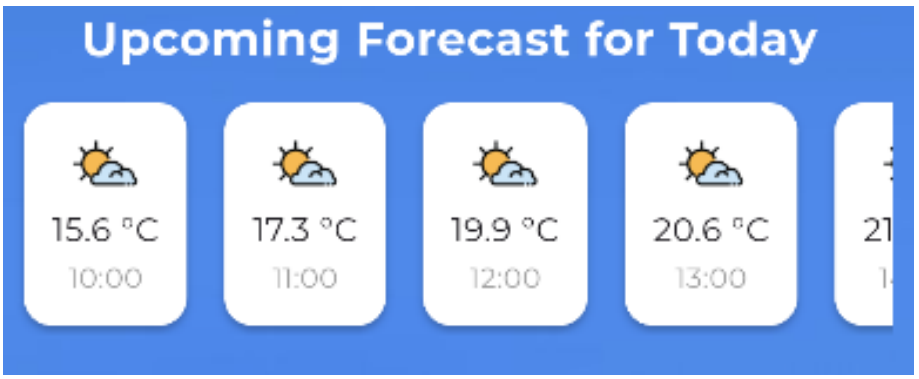


Fig. 4 - Hourly Forecast Row displaying the hourly weather information

If the user clicks on one of those tiles, the weather card displaying the weather info is instantly updated to display the weather data based on that timestamp. For example, if the user clicks on the 12:00 tile, this is how the page looks like:



Fig. 5 - Updated weather card when clicking on a timestamp tile in the “Upcoming Forecast” row

As we can see, the data has been updated and a button “Reset to current hour” appears that lets the user reset the weather data based on the current hour.

If we go down below the same page, we can see two new sections: “Alerts” section, used for displaying current weather alerts like possible floods, etc; “Saved Cities” section where the user can save cities, by searching for them on the search bar and clicking on “Save City”. Users can check on the city's weather by clicking on one of the saved city tiles. The user must insert at least 3 chars in the input field otherwise the submission to save the city won’t be accepted:

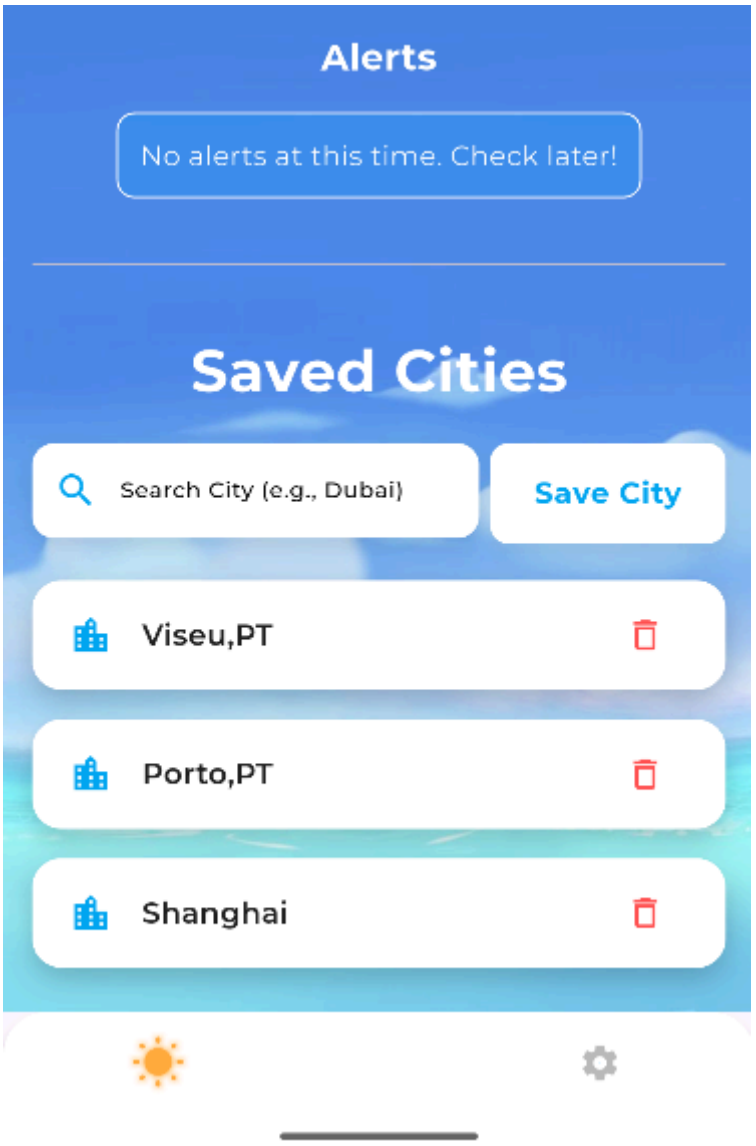


Fig. 6 - Alerts and Saved Cities sections of the Home Page

Next, we display how the autocomplete feature looks like. If we try to search for “Dubai” and type just “Du”, this is what appears:

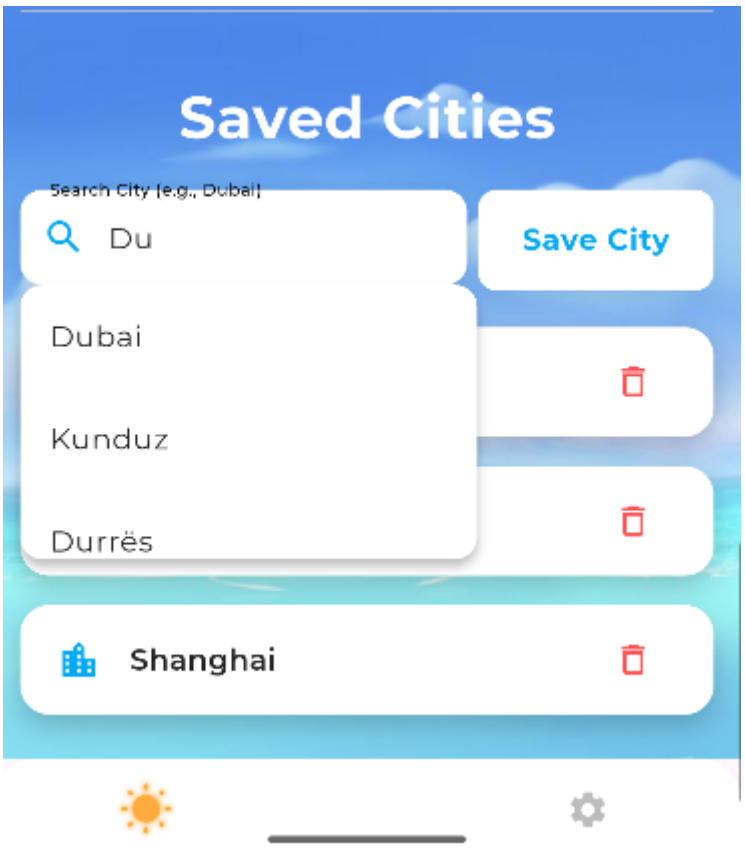


Fig. 7 - Autocomplete text search for saving cities

Throughout our app, we added pop-up messages whenever the user completes important actions in order to provide a better experience and to allow the user to have a perception of the things he is doing. For example, this is the pop-up message that appears when a user adds the city “Dubai” to his saved city list:

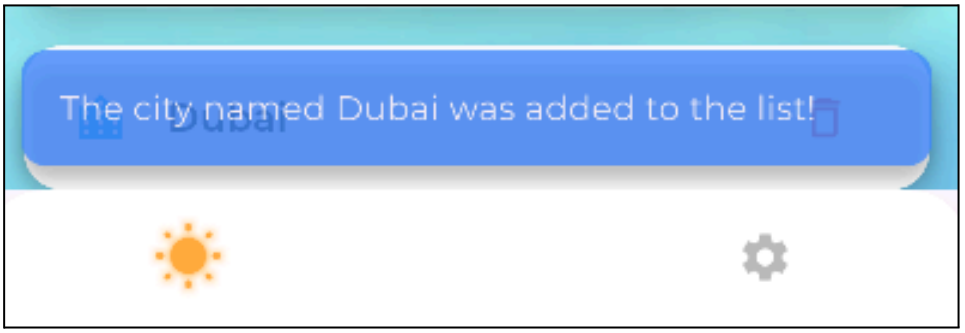


Fig. 8 - Pop-up message

Also, when the user tries to delete a city, or in case the click is made accidentally, we display a confirmation message for the user to confirm his action:

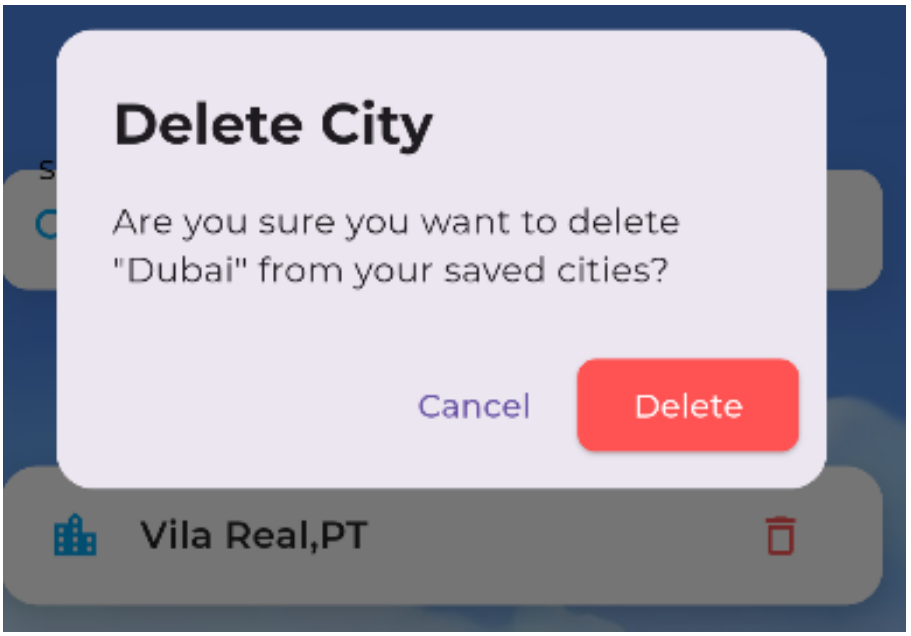


Fig. 9 - Delete confirmation message

If the user clicks on “Delete”, the city is deleted and another pop-up message appears displaying the removal of the selected city. Now, if we click on a city, on this case “Vila Real, PT”, this is what appears:



Fig. 10 - City View Page (Today Section)

As we can see, this page is pretty similar to the Home Page, however, it contains a “Sun Path Tracker” section, to provide a visual representation of when the sun rises and sets:

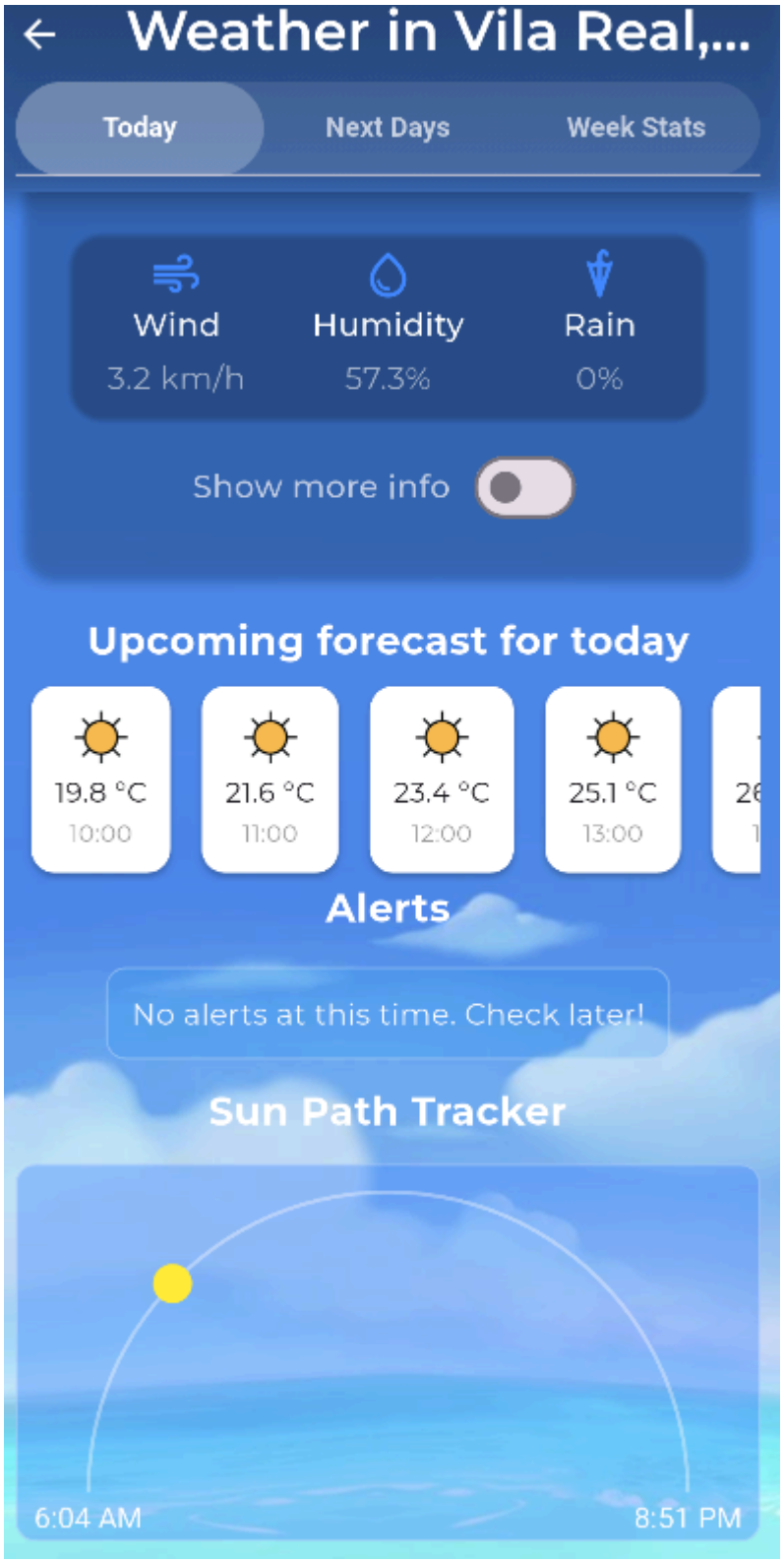


Fig. 11 - Sun Path Tracker to display a visual representation of when the sun rises and sets

Also, on the top, we have now 3 different tabs. The first tab is the one being shown in Fig.10, displaying today's weather of the selected city. The second tab is responsible for displaying the weather on the upcoming days of the week. By default, it displays tomorrow's weather in order for it to be consistent with what was requested in the project proposal, however, we allow the user to select another upcoming day of the week:



Fig. 12 - City View Page (Next Days Section)

Here, the page is almost identical to the Home Page. The only difference is the droplist below the title. If we click on it, this is what we get:

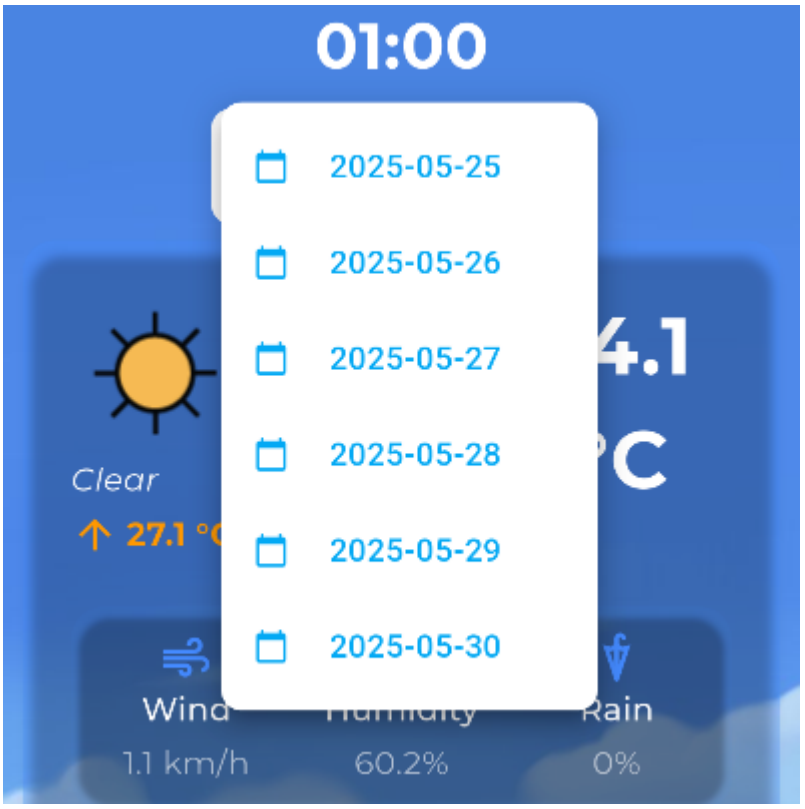


Fig 13 - Droplist for user to pick another day in the “Next Days” section of the City View Page

If we click on a timestamp, to provide a better user experience, the page is updated immediately and in real-time, without requiring to refresh the entire page.

Lastly, when we click on the final “Week Stats” tab, this is what we get:

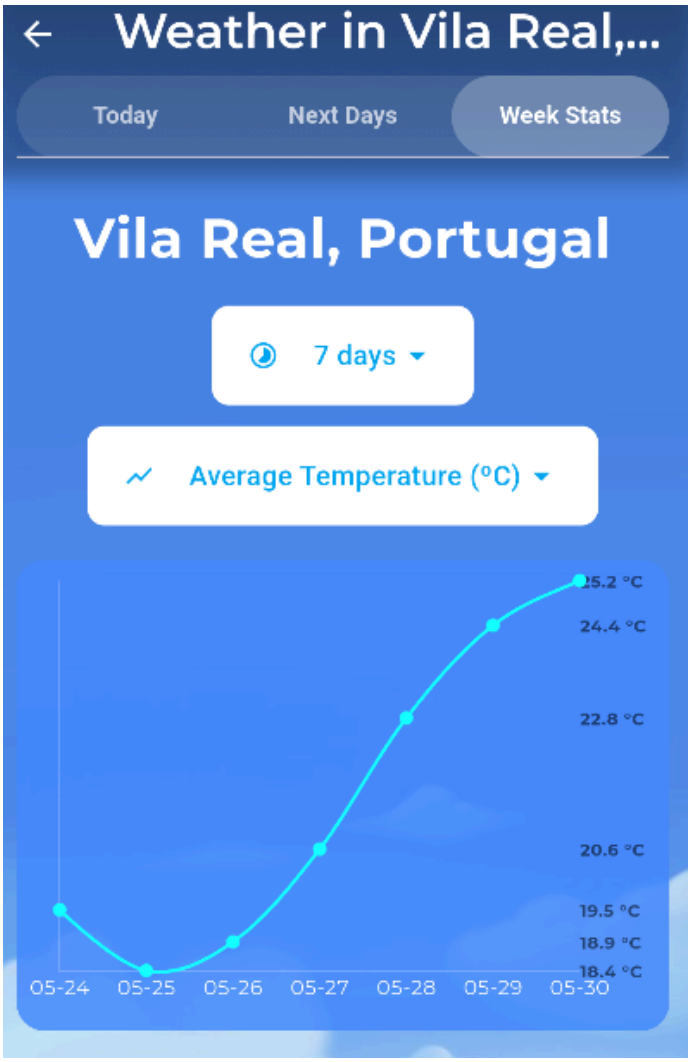


Fig. 14 - City View Page (Week Stats Section)

Here, the user can see a graphical evolution of the current weeks' weather. To give the user some freedom, we allow him to pick the date interval and the weather characteristic he wants to study. When the user changes one of the selected fields, the graph data is also updated in real-time:



Fig. 15 - Droplist to select date range in Week Stats section in City View Page

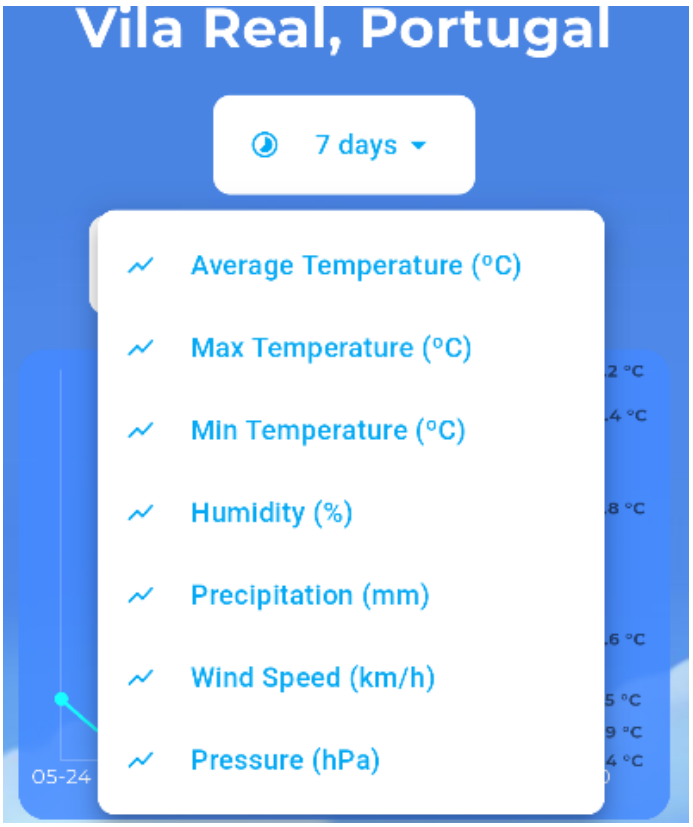


Fig. 16 - Droplist to select the weather characteristic on which the graph data consider

If we go back to the Home Page, by clicking on the back arrow on the top left corner, and we click on the Settings option in the navigation bar, we are directed to the Settings Page. In this project, we tried and went for a user free experience and we think that we really achieved that since, on this page, the user can pick his preferred metric units and select if he wants the switch button, depicted in Fig. 3, to be already enabled when seeing the weather card:

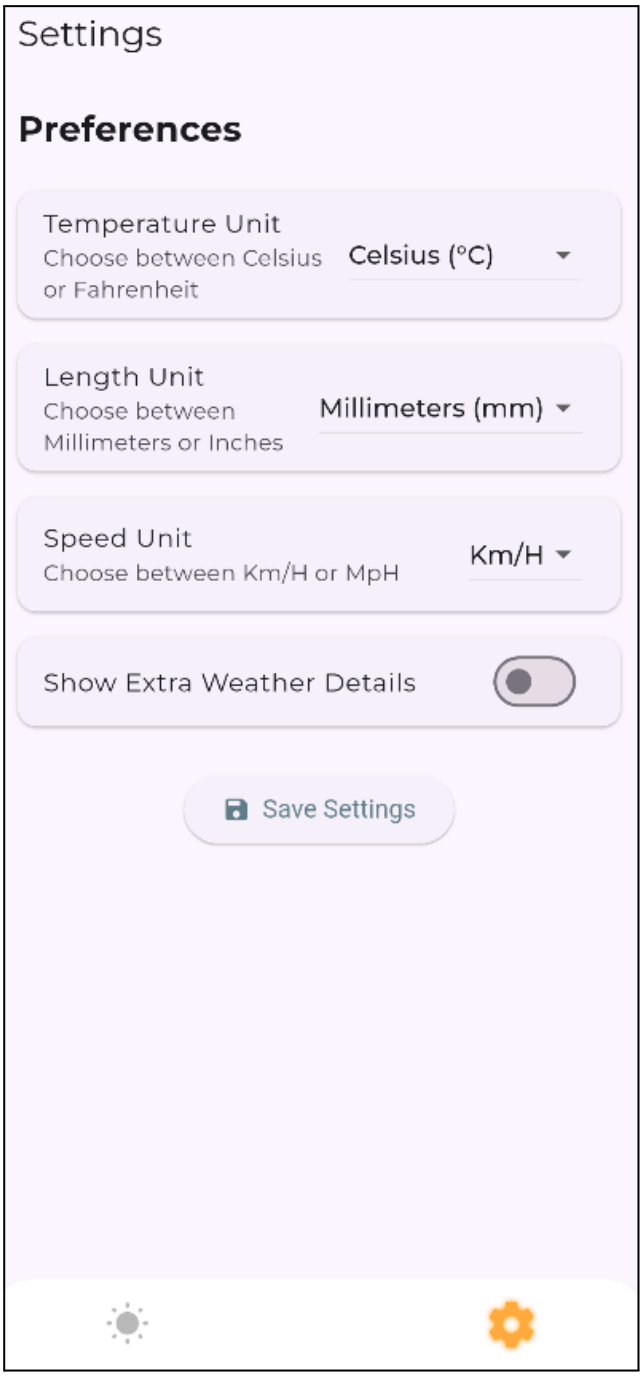


Fig. 17 - Settings Page